

LEVEL II

Surveillance Those Whom Surveillance You

The Image Archive

Miyö Van Stenis | 2015 — 2026

vigipirate.schloss-post.com

First presented at SIOSP, University of Galway, May 2026

SYSTEM ARCHITECTURE

VQD operates in four levels. Each level is autonomous — it functions without the others. Each level is also cumulative — it extends the capacity of what precedes it.

Level I Encrypted Storage — the data that cannot be seized

Level II **The Image Archive — surveillance of those who surveil you**

Level III The Drone — autonomous escape and evidence mobility

Level IV SMS Trigger — the moment of execution

| *This document covers Level II. Begin with Level I if your archive is not yet encrypted.*

PREMISE

The state surveils you. It collects images of your protests, your gatherings, your movements, your face in a crowd. It organizes these images by date, by location, by association. It uses them to designate, to charge, to disappear.

This recipe inverts the operation.

You will build a system that collects images of the state — of its officials, its institutions, its security apparatus, its public violence — organized by year, by political source, with full provenance metadata. The state will continue to surveil you. You will surveil it back.

This is not metaphor. It is a technical protocol. The data you collect becomes an archive. The archive becomes evidence. Evidence is what survives.

The system was first designed in 2015, during a residency at Schloss Post, using a Python script searching Google, Yahoo, DuckDuckGo, and Tor onion search engines. The keywords were drawn from a list of Venezuelan political figures, institutions, and terms of violence. The result was a database of images organized as a counter-narrative — not the narrative the state produces about itself, but the images that accumulate around it from multiple sources simultaneously.

The 2026 version runs five search engines in parallel: Bing, Flickr, Imgur, Naver, and Google. It downloads up to 3,000 images per keyword per session. It operates autonomously, without requiring your presence. The archive grows while you are elsewhere.

INGREDIENTS

HARDWARE

- Any computer running Windows 10/11, macOS 10.14+, or Linux
- Minimum 10GB free storage per year of archive (more is better)
- Stable internet connection
- External hard drive — for Level I encrypted storage of the archive

SOFTWARE

- Python 3.9 to 3.12 — from python.org (not 3.13+)
- VQD_CRAWLER — github.com/miyovanstenis/VQD_CrawlerV2
- VeraCrypt — veracrypt.fr (for Level I, before you begin)
- A text editor — to configure keywords
- Any web browser — to use the configuration editor

OPTIONAL BUT RECOMMENDED

- Flickr API key — free from flickr.com/services/apps/create
- A VPN — do not run the crawler on a network that can be directly attributed to you
- A dedicated USB drive containing the VQD_CRAWLER folder — portable, no installation traces

On the VPN: the crawler makes hundreds of HTTP requests to image search engines. On a hostile network — a university in a country with internet surveillance, a public network in a controlled political environment — this activity is visible. Route it through a VPN before starting.

BEFORE YOU BEGIN

Level II feeds Level I. The images the crawler collects have no value if they can be seized. Before running the crawler, you need an encrypted container to store the output. If you have already completed Level I, your VeraCrypt container is mounted and waiting. If not, stop here and complete Level I first.

The order is not procedural preference. It is political logic. An unencrypted archive of images documenting state violence, collected on a machine that can be seized, is not a counter-surveillance system. It is evidence that can be turned against you.

Checklist before running

- Level I VeraCrypt container mounted and accessible
- VPN active
- VQD_CRAWLER installed and built (see Installation)
- sources.json reviewed and keywords verified for your political context
- Output directory set to your mounted VeraCrypt container

INSTALLATION

Windows

Install Python 3.11 from python.org. During installation, check the box that reads Add Python to PATH. This is mandatory — without it, the build script cannot find Python.

Download VQD_CRAWLER from github.com/miyovanstenis/VQD_CrawlerV2. Extract the folder. Open it. Double-click BUILD.bat.

The script will:

- Locate your Python installation automatically

- Install all required libraries
- Compile VQD_CRAWLER.exe
- Open the output folder when finished

Run VQD_CRAWLER.exe from inside the dist/VQD_CRAWLER/ folder. Do not move the exe out of that folder — it depends on files in the same directory.

If BUILD.bat closes immediately without running, open PowerShell in the vqd_crawler folder (Shift + right-click in empty space, then Open PowerShell here) and type .\BUILD.bat — the error message will be visible.

macOS / Linux

Install Python 3.11. On macOS: brew install python@3.11. On Ubuntu: sudo apt install python3.11 python3.11-tk.

Download the repository. Open a terminal in the vqd_crawler folder. Run:

```
pip3 install -r requirements.txt
python3 VQD_CRAWLER_GUI.py
```

Or build a standalone application:

```
chmod +x build.sh && ./build.sh
```

Python version constraint

The build system requires Python 3.9 to 3.12. Python 3.13 and above are not currently supported. If you have installed a newer version, install 3.11 alongside it — Python supports multiple versions on the same machine. The build scripts will find the correct version automatically.

CONFIGURATION

The crawler is driven by a single file: sources.json. This file defines which keywords to search, in which years, from which robot (gobierno or oposicion), using which search engines. Every aspect of what the crawler collects is determined by this file.

You do not need to edit JSON directly. Open VQD_CONFIG_EDITOR.html in any browser. This is a standalone configuration editor — no internet connection required, no installation, no account. Edit your keywords, your search jobs, your interface colors. Export the updated sources.json. Replace the file in your VQD_CRAWLER folder. The crawler will use the new configuration immediately, without rebuilding.

The two robots

The archive is organized around two investigative positions: gobierno and oposicion. This binary is not a political endorsement of either position — it is a methodological structure. Two robots search simultaneously for the same keyword set from opposite institutional perspectives. The resulting archive contains both the state's self-representation and the opposition's documentation of the state. Neither is complete. Neither is neutral. The archive holds both.

In a different political context — a different country, a different conflict — the robot names can be changed in sources.json to reflect the specific power structure being documented. The binary structure remains: those who hold power and those who resist or document it.

The keyword logic

The original 2015 VQD keyword list was built from political figures, institutions, and terms of violence specific to Venezuela. The 2026 version extends this list to include figures from the post-2019 crisis, the 2024 election fraud, and the current period of post-electoral repression.

To adapt this system to a different political context:

1. Open VQD_CONFIG_EDITOR.html in your browser
2. Navigate to the Keywords tab

3. Remove existing figures that do not apply to your context
4. Add names of government officials, security forces, institutions
5. Add names of opposition figures, human rights defenders, journalists
6. Add institutional names: ministries, police bodies, courts, prisons
7. Add terms of violence specific to your context: names of detention centers, legal designations used to criminalize dissent, names of state security units
8. Click Generate Search Jobs from Keywords — the system will build all search combinations automatically
9. Export sources.json and replace the file in your VQD_CRAWLER folder

Years

The crawler organizes images by year. The default configuration covers 2014 to 2025. To add years beyond this range, open VQD_CONFIG_EDITOR.html, navigate to Search Jobs, click Add Job, and set the year and query. Or edit sources.json directly — each job is a JSON object with robot, year, query, and sources fields.

Search engines

The crawler uses five search engines simultaneously:

- Bing — the primary source. Returns up to 1,000 images per query. The most consistent results across political content.
- Flickr — photojournalism archive. The most useful source for protest photography, documentation of public events, and images with clear provenance. Requires a free API key for maximum results.
- Imgur — community image repository. Useful for content that has circulated widely outside mainstream media channels.
- Naver — South Korean search engine with up to 3,000 results per query. Covers international news archives with different editorial perspectives than Western search engines. Particularly useful for content that has been removed or suppressed in Western-indexed sources.
- Google — 100 images per query, highest relevance filtering.

All five engines run in parallel. 15 simultaneous download connections. The minimum file size is 30KB — icons, thumbnails, and low-resolution images are discarded automatically.

RUNNING THE CRAWLER

GUI mode

Launch VQD_CRAWLER.exe (Windows) or python3 VQD_CRAWLER_GUI.py (all platforms). The interface shows three tabs:

- Images — thumbnail grid of everything collected in the selected robot/year folder. Click any year in the left panel to view its archive.
- Settings — configure robot, year, query, search engines, and output directory before running.
- Log — live output of the crawler: which images were saved, from which engine, at what file size.

Set the archive directory (Settings tab, Archive dir field) to your mounted VeraCrypt container. This is the most important setting — it determines where the images are stored.

Click RUN to start a single search session. Click Run sources.json to run all configured jobs in sequence — this is the full batch mode and will run for several hours depending on the number of keywords and years configured.

Command line mode

For automated operation — running the crawler on a schedule, or on a machine without a graphical interface — use the command line directly:

```
python crawler.py --robot gobierno --year 2024 --query "Nicolas Maduro Venezuela"
python crawler.py --config sources.json
python crawler.py --stats
```

The `--config` flag runs all jobs defined in `sources.json` in sequence. The `--stats` flag prints a summary of the current archive without downloading anything.

Session behavior

The crawler is resumable. Every downloaded image is logged immediately to `metadata.txt` in its year folder. If a session is interrupted — by a network failure, by power loss, by you — the next session reads the metadata log and skips images already downloaded. You do not lose what has already been collected.

The metadata log records for each image: timestamp, filename, source engine, search query, and original URL. This is the provenance record. It is what allows the archive to function as evidence rather than as an unattributed image collection.

Do not delete metadata.txt files. They are the archive's memory. Without them, the deduplication system does not know what has already been collected, and the crawler will re-download the same images in every session.

What to expect

For a single search query — for example, 'Maria Corina Machado Venezuela 2024' across Bing, Flickr, Imgur, and Naver — the crawler will typically collect between 200 and 800 images in a single session, depending on network speed and the availability of results for that keyword.

For a full batch run of all 74 configured jobs across 12 years and two robots, expect the session to run for 8 to 24 hours. Run it overnight. Point the output to your VeraCrypt container. Leave it.

The archive that results is not curated. It contains maps, infographics, propaganda materials, protest photographs, news images, social media screenshots, and content with no visible relationship to the query. This is correct. Curation is a later operation. The first function of the crawler is accumulation — the construction of a visual mass that the state did not intend to produce but which its own image-producing activity generates as a byproduct.

VIEWING THE ARCHIVE

The crawler includes a gallery generator that produces a browsable HTML index of the archive, organized by year. This replicates the localhost directory display of the original 2015 VQD installation at Schloss Post.

```
python vqd_gallery.py
python vqd_gallery.py --serve
```

The `--serve` flag launches a local web server at `localhost:8000` and opens the gallery in your browser. The gallery shows all images across both robots for each year, shuffled randomly — so each reload of the page produces a different sequence from the same archive. This is the behavior described in the original Schloss Post documentation: Reload this page to see more images.

The gallery is a tool for reading the archive, not a public display. It runs locally. It requires no internet connection. It does not transmit the archive anywhere.

OPERATIONAL SECURITY

Building a counter-surveillance archive is not a neutral activity. If you are collecting images that document state violence in a country with internet surveillance, treat the crawler as an operational system, not a research tool. The difference is the threat model.

Network

- Run the crawler behind a VPN. Select a VPN provider outside the jurisdiction of the state you are documenting.
- Do not run the crawler on institutional networks — university networks, workplace networks, networks that can be attributed to you by name.
- If you are in the country you are documenting, use Tor in addition to a VPN for the highest risk sessions.

Storage

- The output must go directly into a VeraCrypt container (Level I). Do not store unencrypted images on your main drive, even temporarily.
- Distribute the encrypted container across at least two geographic locations. Level I documents this in detail.
- Do not keep the VeraCrypt container mounted when you are not actively using it.

Attribution

- Do not run the crawler from accounts or machines registered in your name.
- Do not commit the archive to public repositories.
- The VQD_CRAWLER code is open source. The archive it produces is not.

The keyword list as a security consideration

The keywords configured in `sources.json` determine what your crawler searches for. If your machine is seized, `sources.json` reveals your research focus. Keep `sources.json` inside the VeraCrypt container, or configure it to contain only generic terms on the main drive, with the specific political keyword list stored encrypted separately.

The original 2015 script was run from a machine under a pseudonym on a network that was not directly attributable. The archive it produced was stored on an encrypted drive. The split between Miyö Van Stenis and Anaïs Garza — between the artist name and the legal name — operated as operational security before it was understood as such. Build that split into your system deliberately.

WHAT THIS SYSTEM DOES

The crawler does not produce truth. It produces accumulation. The images it collects are not verified, not curated, not interpreted. They are timestamped, sourced, and stored.

The political function of the system is not revelation — it is preservation. The state erases. Search engines delist. Social platforms remove. News archives disappear behind paywalls or are deleted outright. The crawler operates before the erasure, building an archive of what existed at a specific historical moment.

Ariella Azoulay argues that to preserve is to refuse the colonial and state logic of erasure — that the archive says: the destruction has not yet succeeded. This is the claim Level II makes. Not: here is the truth about Venezuela. But: here is what was visible, on these dates, from these sources, under these keywords. The destruction has not yet succeeded.

Grégoire Chamayou describes the drone as a machine that makes the target permanently legible — permanently targetable — from a distance, with zero risk to the sovereign. VQD inverts this logic at Level II. The crawler makes the state permanently legible — from a distance, from wherever you are, with minimal risk to yourself. The same logic. The opposite direction.

The crawler watches the state back. This is the recipe.

TECHNICAL REFERENCE

Search engine parameters

Bing	1,000 results / 35 per request / offset 1 to 1000
Flickr	Unlimited with API key / 100 per page
Imgur	5,000 results / page-based crawl
Naver	3,000 results / 100 per request
Google	100 results via icrawler

Parallel connections

15 simultaneous download threads. Request delay 0.3 seconds per thread. Minimum file size 30KB. Invalid image files (non-readable by PIL) are discarded automatically.

File structure

```
archive/
  gobierno/
    2015/
      image_001.jpg
      image_002.png
      metadata.txt
  oposicion/
    2015/
      ...
```

metadata.txt format

```
2026-05-11T14:30:22Z | filename.jpg | bing | query string | https://source
```

Command reference

```
python crawler.py --robot gobierno --year 2015 --query "..."  
python crawler.py --robot oposicion --year 2017 --sources bing flickr imgur  
python crawler.py --config sources.json  
python crawler.py --site https://minci.gob.ve --depth 2 --robot gobierno --year 2016  
python crawler.py --stats  
python vqd_gallery.py --serve
```

VQD Level II | Vigipirate Quadcopter Drone: A Counter-Surveillance Cookbook

Miyö Van Stenis | GNU GPL v3 | github.com/miyovanstenis/VQD_CrawlerV2